SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA0305	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	M.Manoj Kumar	**Reg. No.:**	192425114
Section / Batch:	B. Tech - AI & ML	**Date:**	16-08-2026

Code of Conduct

I M.Manoj Kumar (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _M. Manoj Kumar_____

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.
Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.
Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.
Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.
Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Q1. Context-Free Grammar Trees vs Dependency Parsing

Introduction

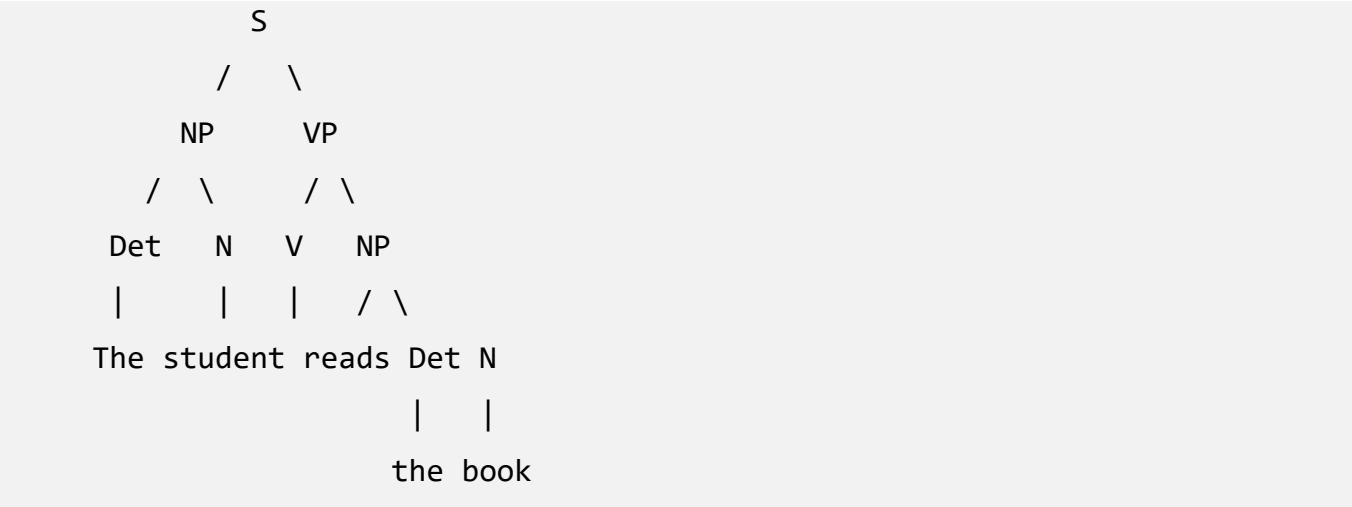
A language processing system needs an effective representation of sentence structure for grammar checking, syntactic analysis, and understanding relationships between words. CFG primarily represents a sentence through hierarchical constituent structure, while dependency parsing represents the sentence through direct relationships between words. The choice depends on whether the system cares more about phrases and grammatical constituents, or about relationships between individual words.

1. Context-Free Grammar Representation

A CFG consists of production rules describing how larger linguistic units are built from smaller units. For example:

```
S → NP VP
NP → Det N
VP → V NP
Det → the
N → student
V → reads
```

For “The student reads the book,” a CFG represents the sentence hierarchically:



“The student” and “the book” each form noun phrases, so CFG is useful when the system needs to understand constituents.

2. Dependency Parsing Representation

Dependency parsing instead establishes relationships between individual words. For the same sentence:

```
          reads
         /     \
      student    book
        |         |
       The       the
```

Here reads is the root, student is the subject, book is the object, and The/the depend on their respective nouns. The focus is on head-dependent relationships rather than phrase grouping.

3. Major Difference

Aspect	**CFG Trees**	**Dependency Parsing**
Basic representation	Constituents/phrases	Word-to-word relations
Structure	Hierarchical tree	Dependency tree
Main unit	Phrase	Word
Focus	Phrase structure	Grammatical relationships
Example	NP, VP, PP	Subject, object, modifier
Grammar rules	Explicit production rules	Dependency relations
Interpretation	Good for constituent analysis	Good for relational analysis
Computational use	Grammar checking, syntax analysis	Information extraction, semantic processing

4. Capturing Relationships Between Words

Dependency parsing is particularly strong when the goal is to identify which word relates to which other word. For “The researcher developed a model,” dependency analysis directly represents:

```
developed
   |
   ├── researcher → subject
   |
   └── model → object
```

This is useful for information extraction, question answering, semantic role analysis, relation extraction, and knowledge graph construction — e.g. the triple researcher → developed → model can be converted directly into a structured representation.

5. Advantages of CFG

CFG clearly identifies constituents such as noun, verb, prepositional and adjective phrases, which makes it useful for syntactic analysis. Because its rules explicitly specify valid structures, it is well suited to grammar checking, and its hierarchical trees are intuitive and important for describing formal grammatical structure in theoretical linguistics.

6. Limitations of CFG

A basic CFG focuses on constituent structure and does not directly encode semantic relationships between words. It may also need a large number of rules to capture linguistic variation, and ambiguity can produce multiple valid parse trees for the same sentence. For example, in “I saw the man with a telescope,” the phrase “with a telescope” may modify the act of seeing or the man, and a CFG may generate more than one possible tree without indicating which is more likely.

7. Advantages of Dependency Parsing

Dependency parsing directly connects words according to grammatical dependencies, giving a compact representation without intermediate phrase nodes such as NP and VP. It is particularly useful for machine translation, information extraction, search systems, question answering, and sentiment analysis. It is also well suited to languages with relatively flexible word order, since grammatical relationships are represented independently of strict constituent order.

8. Limitations of Dependency Parsing

Dependency parsing does not explicitly represent constituent boundaries the way CFG does, and some linguistic phenomena are easier to describe using phrase structure. It can also become challenging with coordination, ellipsis, discontinuous structures, and complex long-distance dependencies.

9. Which Is More Effective?

CFG is excellent for answering “How are words grouped into phrases?” while dependency parsing is better for answering “How are these words grammatically related?” For the specific requirement of capturing relationships between words, dependency parsing is generally more effective; CFG remains valuable when the priority is grammar checking and syntactic structure.

Conclusion

CFG and dependency parsing are not complete replacements for each other. CFG provides strong hierarchical constituent information, while dependency parsing provides direct relational information. For modern NLP systems that need to extract relationships and support downstream semantic applications, dependency parsing is generally the more suitable representation, while CFG remains important for constituent-based grammatical analysis.

Q2. Top-Down Parsing vs Earley Parsing

Introduction

This scenario describes a real-time application that must process user input efficiently even when sentences are incomplete or ambiguous. The existing system uses top-down parsing, while Earley parsing is being considered as an alternative — an important comparison because real-time systems frequently receive incomplete input such as “I want to book a...” and must still process it usefully.

1. Top-Down Parsing

Top-down parsing begins with the grammar’s start symbol and attempts to derive the input sentence:

```
S
↓
NP VP
↓
Det N VP
↓
The student VP
↓
The student reads
```

Basic process:

```
Start Symbol
     ↓
Grammar Rule Selection
     ↓
Expand Non-terminals
     ↓
Match Input
     ↓
Complete Parse
```

2. Advantages of Top-Down Parsing

It is simple to understand and implement, and its goal-directed approach can be efficient for simple, predictable grammars. In a restricted domain — for example a command system accepting Book + [flight] + [destination] — top-down parsing can work well.

3. Limitations of Top-Down Parsing

Top-down parsing can struggle with ambiguity and incomplete input. For “The student...” the parser expects S → NP VP; after the noun phrase it still expects a VP, so if the user pauses, the parser has not completed the sentence. It can also require backtracking: if the wrong grammar rule is selected, the parser must return to an earlier decision and try another, increasing computational cost.

4. Earley Parsing

Earley parsing is a general algorithm designed to handle a broad class of context-free grammars. It maintains a set of partial parsing states as it processes the sentence. A simplified Earley state is written as:

```
A → α • β
```

The dot shows how much of the production has been recognized — for example, VP → V • NP means the verb has been recognized and a noun phrase is now expected.

5. Main Operations of Earley Parsing

Earley parsing uses three operations: Prediction (predicting possible productions for an expected non-terminal), Scanning (matching an expected terminal with the next input word), and Completion (updating states waiting on a production once it is fully recognized).

```
Input
  ↓
Prediction
  ↓
Scanning
  ↓
Completion
  ↓
More states
  ↓
Final parse
```

6. Handling Incomplete Input

This is where Earley parsing becomes particularly useful. For “I want to book a,” the parser does not have to fail simply because the sentence is incomplete — it can maintain partial states such as:

```
S → NP VP •
VP → V NP •
NP → Det • N
```

This makes Earley parsing well suited to autocomplete, conversational systems, interactive NLP, speech interfaces, and real-time language processing generally.

7. Comparison

Aspect	**Top-Down Parsing**	**Earley Parsing**
Strategy	Expand grammar rules	Dynamic programming
Ambiguity	Can require backtracking	Can maintain multiple possibilities
Incomplete input	Less flexible	More suitable
General CFG support	Depends on parser design	Yes
Partial parsing	Limited	Strong
Implementation	Relatively simple	More complex
Real-time dynamic input	Less suitable	More suitable
Efficiency	Good for restricted grammar	Strong for general CFGs

8. Why Earley Is Better for the Scenario

The scenario emphasizes real-time processing, incomplete input, and ambiguity. Earley parsing addresses these better because it maintains partial hypotheses rather than requiring a complete sentence before useful analysis takes place. For “Can you find me a hotel in...,” a flexible parser can hold the partial structure and wait for more input, whereas a top-down parser needs more explicit grammar handling and potentially backtracking.

9. Computational Considerations

Earley parsing has a worst-case complexity of approximately $O(n^3)$ for general context-free grammars, though it performs better in practice for many grammars. Top-down parsing can be very efficient for small, deterministic, carefully designed grammars, so Earley is not always faster — it provides better

flexibility for general, ambiguous, and incomplete input, while top-down parsing can be more efficient for simple, predictable grammars.

Conclusion

For the given real-time application, Earley parsing is the more suitable choice because the system must handle incomplete and ambiguous input dynamically. Top-down parsing remains useful for constrained applications, but Earley's ability to maintain partial parsing states makes it better suited to interactive language understanding.

Q3. CFG vs PCFG vs Neural Parsing for Ambiguous Sentences

Introduction

This question considers “She saw the man with a telescope,” which is structurally ambiguous: “with a telescope” could mean she used a telescope to see the man, or the man had a telescope. The three approaches compared are CFG, PCFG, and neural parsing.

1. CFG Approach

A standard CFG describes possible grammatical structures through production rules:

```
S → NP VP
VP → V NP
VP → VP PP
NP → NP PP
```

Interpretation 1: She [saw the man] [with a telescope] — the PP modifies the verb phrase, meaning she used the telescope.

Interpretation 2: She [saw [the man with a telescope]] — the PP modifies the noun phrase, meaning the man had the telescope.

CFG can represent the ambiguity but does not by itself determine which interpretation is more likely.

2. Advantages and Limitation of CFG

CFG offers a simple formal representation, explicit grammatical rules, and easy interpretability, and it is useful for theoretical syntax. Its main limitation is that standard CFG is not probabilistic — if two parses are grammatically valid, it provides no statistical way to choose the more likely one.

3. PCFG

A Probabilistic Context-Free Grammar (PCFG) extends CFG by attaching probabilities to grammar rules, for example:

```
VP → VP PP       0.30
NP → NP PP       0.20
```

If a training corpus shows VP attachment = 0.70 and NP attachment = 0.30, the parser can prefer the VP interpretation. Instead of asking “Is this parse grammatically possible?”, PCFG asks “Which grammatical parse is more probable?”

4. Advantages and Limitations of PCFG

PCFG can rank competing interpretations using probabilities learned from annotated corpora, making it more practical than plain CFG. However, its accuracy depends heavily on the grammar and training data, and traditional PCFGs often assume rule independence, so they can struggle with long-distance relationships, rich semantic context, and complex lexical information.

5. Neural Parsing

Neural parsers use machine-learning architectures such as RNNs, Transformers, or pretrained language models, learning representations from data rather than relying only on hand-specified rules. For “She saw the man with a telescope,” a neural parser can use contextual information from the whole sentence — the representation of “with a telescope” is influenced by surrounding words like saw, man, telescope, and she — allowing it to learn patterns associated with different attachment decisions.

6. Comparison

Feature	**CFG**	**PCFG**	**Neural Parsing**
Probability	No	Yes	Yes, via model scores
Ambiguity handling	Generates alternatives	Ranks alternatives	Predicts likely structure
Context sensitivity	Low	Moderate	High
Training requirement	Grammar design	Annotated corpus	Usually large datasets
Interpretability	High	High–moderate	Lower
Generalization	Limited by grammar	Better	Strong
Real-world performance	Limited	Better	Usually strongest

7. Why Neural Parsing Is Effective

Ambiguity resolution often depends on context. “She was observing distant objects. She saw the man with a telescope” strongly suggests she used the telescope, while “The man carrying scientific equipment had a telescope. She saw the man with a telescope” makes the noun-attachment reading more plausible. A purely rule-based CFG cannot naturally use this broader context; PCFG improves on CFG with probabilities, but neural models can incorporate much richer contextual representations.

8. Real-World Trade-Off

CFG is best when the grammar is small, explainability is important, and the domain is highly controlled. PCFG is best when a statistical ranking mechanism is needed, annotated data is available, and interpretability still matters. Neural parsing is best when large datasets and computational resources are available and complex linguistic context and high accuracy are required.

Conclusion

CFG can identify multiple possible structures but cannot effectively rank them. PCFG improves on this by assigning probabilities to alternative parses. Neural parsing goes further by learning contextual representations from broader linguistic information. For real-world NLP applications, neural parsing is generally the most effective of the three when sufficient data and compute are available, though PCFG and CFG remain valuable for their interpretability and explicit grammatical structure.

Q4. Feature Structures vs Subcategorization Frames

Introduction

This scenario requires a grammar system to correctly handle subject–verb agreement and verb argument structures. The two approaches, feature structures and subcategorization frames, solve related but different linguistic problems.

1. Feature Structures

A feature structure represents linguistic information using attributes and values, for example:

```
[NUMBER: singular
 PERSON: third
 TENSE: present]
```

For “The boy runs,” the subject has NUMBER = singular, PERSON = third, and the verb must agree with matching features, allowing the grammar to enforce agreement.

2. Subject–Verb Agreement

“The boy runs” is correct, but “The boy run” is not. A feature-based grammar represents NUMBER = singular, PERSON = third on the NP and requires the VP or verb to carry compatible features. For “The boys run,” the subject becomes NUMBER = plural, and the verb must match — one of the major strengths of feature structures.

3. Feature Unification

Feature structures are often used with unification: if the subject has NUMBER = singular and the verb has NUMBER = plural, the structures cannot unify and the sentence can be rejected as grammatically inconsistent. When both sides agree, unification succeeds, giving a systematic way to enforce grammatical constraints.

4. Subcategorization Frames

A subcategorization frame describes what arguments a verb requires or permits. Different verbs have different argument structures:

```
eat  → Subject + Object            (“The student eats food.”)
sleep → Subject                     (“The student sleeps.”)
```

```
give → Subject + Object + Indirect Object
        (“The teacher gives the student a book.”)
```

Subcategorization therefore focuses primarily on the arguments a predicate expects.

5. Why Argument Structure Matters

“The student gave” feels incomplete because give typically expects additional arguments, as in “The student gave the teacher a book” (subject = student, indirect object = teacher, direct object = book). Subcategorization frames help a parser determine whether the required arguments are present.

6. Main Difference

Aspect	**Feature Structures**	**Subcategorization Frames**
Main purpose	Represent linguistic attributes	Represent verb argument requirements
Agreement	Strong support	Not the main purpose
Number / Person / Tense	Yes	Not primary
Argument structure	Can represent	Core purpose
Unification	Central mechanism	May be incorporated
Scope	Broad linguistic constraints	Mainly predicate-argument structure

7. Example Combining Both

For “The students gave the teacher books,” a feature-based representation specifies Subject NUMBER = plural, PERSON = third, which the verb must agree with. At the same time, the subcategorization frame of give specifies Subject + Indirect Object + Direct Object, so students → Subject, teacher → Indirect Object, and books → Direct Object. A complete NLP grammar may therefore use both techniques together.

8. Strengths and Limitations of Feature Structures

Feature structures are highly effective for agreement constraints and can represent number, person, gender, case, tense, aspect and mood, enforcing compatibility between grammatical components across constructions. Their main limitation is that they can become complex and hard to maintain when many interacting linguistic properties are represented.

9. Strengths and Limitations of Subcategorization Frames

Subcategorization frames are excellent for representing predicate argument requirements (e.g. sleep → NP, eat → NP NP, give → NP NP NP), which is important for semantic role labeling, parsing, information extraction and lexical semantics. However, they alone are not sufficient to enforce agreement — they can specify that give needs three arguments, but not whether “The boy gives...” or “The boys give...” has correct agreement. That is where feature structures are stronger.

Conclusion

For subject–verb agreement specifically, feature structures provide better support because they explicitly encode grammatical features and allow them to be unified. For verb argument structures, subcategorization frames are more specialized and effective. If asked which provides better overall support for grammatical correctness, feature structures are the stronger choice, particularly because they represent multiple interacting constraints — though a robust NLP grammar would ideally combine both.

Q5. Transition-Based Parsing vs Graph-Based Parsing

Introduction

This scenario concerns a large-scale NLP system that needs fast and accurate dependency parsing on big datasets. The two alternatives, transition-based and graph-based parsing, differ significantly in how they make parsing decisions.

1. Transition-Based Parsing

Transition-based parsing constructs a dependency tree incrementally through a sequence of actions, maintaining a stack, a buffer, and a set of dependency relations. Typical transitions are SHIFT, LEFT-ARC, RIGHT-ARC and REDUCE.

2. Example

For “She reads books,” a simplified process shifts She and reads onto the stack, creates a dependency between reads and She, shifts books, then creates reads → books, producing:

```
      reads
      /   \
    She   books
```

The parser makes local decisions sequentially.

3. Advantages of Transition-Based Parsing

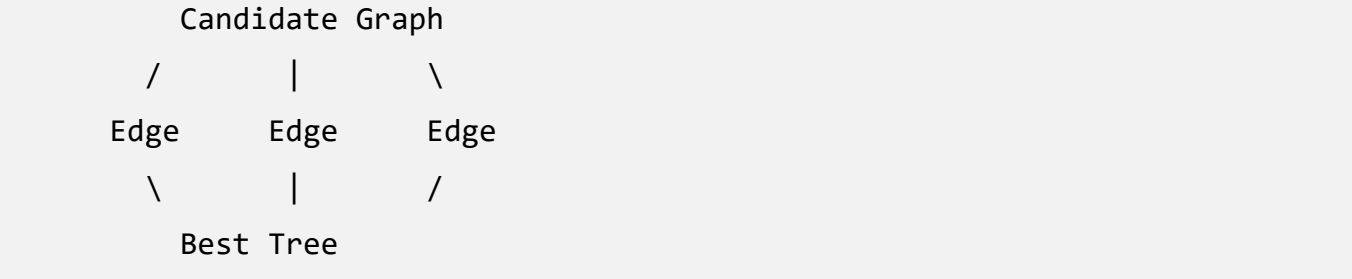
It is generally fast because it makes a sequence of local decisions, typically operating close to linear time O(n) under standard transition systems. This low computational complexity and low latency make it well suited to large datasets and real-time applications.

4. Limitations of Transition-Based Parsing

The main limitation is error propagation from greedy, locally guided decisions: an incorrect dependency identified early can influence later decisions, so a fast decision at an early stage can sometimes cause errors later in the parse.

5. Graph-Based Parsing

Graph-based parsing instead considers possible dependency relationships as a graph over all word pairs, and searches for the dependency tree with the highest overall score rather than committing to a sequence of local moves.



6. Global Decision-Making

Instead of asking “What is the best next action?”, graph-based parsing asks “What is the best complete dependency tree?” which can reduce some of the error-propagation problems associated with sequential parsing.

7. Advantages and Limitations of Graph-Based Parsing

Because the entire tree is considered, graph-based parsing can achieve strong global optimization, better structural consistency, and rich feature modeling. Its primary limitation is computational cost: evaluating many possible dependency relationships requires more processing time, memory, and resources, making it less attractive when extremely high throughput is required.

8. Comparison

Aspect	**Transition-Based Parsing**	**Graph-Based Parsing**
Decision style	Sequential/local	Global
Typical complexity	Approximately O(n)	Usually higher
Speed	Very high	Lower
Memory	Generally lower	Generally higher
Error propagation	More susceptible	Less, from global decisions
Global structure	Limited	Strong
Real-time use	Excellent	Less suitable
Large-scale processing	Excellent	Computationally expensive

9. Performance on Big Datasets

For a workload such as parsing 100 million sentences, even small differences in per-sentence processing time become significant. A fast, low-memory transition-based parser can provide much higher throughput:

```
Large Dataset
      ↓
Transition-Based Parser
      ↓
Fast Sequential Decisions
      ↓
High Throughput
```

This makes it attractive for search engines, large document processing, web-scale NLP, real-time applications, and massive text corpora.

10. Accuracy vs Speed

The central trade-off is transition-based speed and efficiency (with possible local error propagation) versus graph-based global optimization (at higher computational cost). There is no universal winner: if the system needs maximum throughput and low latency, transition-based parsing is usually preferable; if it needs more globally optimized dependency structures, graph-based parsing can be preferable.

11. Suitability for Large-Scale Applications

Because the question specifically requires fast and accurate dependency parsing on big datasets, and large-scale systems process enormous quantities of text, computational efficiency is extremely important. Transition-based parsing is therefore generally the more practical choice, and modern neural transition-based parsers can also achieve strong accuracy, narrowing the traditional gap between speed and accuracy.

Conclusion

For large-scale NLP applications, transition-based parsing is generally more suitable when speed, scalability, and low computational cost are major requirements. Graph-based parsing provides a stronger global perspective and highly coherent dependency structures, but its computational cost can become significant at scale. A practical production system may therefore select a transition-based neural parser when millions or billions of sentences must be processed efficiently.

Overall Comparative Summary

The five questions in the assessment cover different layers of NLP parsing and linguistic representation.

Q	Comparison	Main Strength	More Suitable Approach
Q1	CFG vs Dependency Parsing	Word relationships	Dependency Parsing
Q2	Top-Down vs Earley Parsing	Incomplete/ambiguous input	Earley Parsing
Q3	CFG vs PCFG vs Neural Parsing	Ambiguity resolution	Neural Parsing
Q4	Feature Structures vs Subcategorization Frames	Grammatical correctness	Feature Structures
Q5	Transition-Based vs Graph-Based Parsing	Large-scale efficiency	Transition-Based Parsing

Key Takeaway

There is no single parsing technique that is best for every NLP problem. CFG is strong for constituent structure; dependency parsing is strong for direct word relationships; Earley parsing is useful for incomplete and ambiguous input; PCFG adds probability-based ambiguity resolution; neural parsing provides powerful contextual interpretation; feature structures enforce grammatical constraints such as agreement; subcategorization frames model predicate-argument requirements; transition-based parsing emphasizes speed and scalability; and graph-based parsing emphasizes global structural decisions.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ANSWERS